

CFMOTO RIDE to GPX

A complete, repeatable Windows guide for extracting your own CFMOTO historical ride track and converting it into a clean GPX file

What this guide accomplishes

It reproduces the method that successfully extracted an actual CFMOTO RIDE historical track from the Android app running in BlueStacks and converted it into a GPX track that imported correctly into onX Offroad.

Important: This guide intentionally includes only the working path. It does not use mitmproxy, HAR capture, rooting, private-app storage access, or screenshot tracing. The successful data source is Android **logcat**, because the tested CFMOTO app writes its API request/response information - including the ride trajectory - into the Android system log.

Use this only with an account and ride data you are authorized to access. The captured log can contain account tokens, vehicle identifiers, locations, and other private information. Treat the log file as sensitive and do not post it publicly.

Tested workflow: Windows 10/11 + BlueStacks 5 Android 11 + CFMOTO RIDE + BlueStacks ADB + Android logcat.

Official links

[Download BlueStacks 5](#)

[BlueStacks: create an Android 11 instance](#)

[CFMOTO RIDE on Google Play](#)

[onX Offroad GPX import instructions](#)

Revision: September 2026

1. Overview - What We Are Doing

CFMOTO RIDE stores historical rides in CFMOTO's online service. When a historical ride is opened, the app requests the ride detail and receives a trajectory. In the tested Android build, the app's own logging prints the request and response into Android logcat. We capture that log while opening one ride, then extract the trajectory records and write their longitude/latitude values into GPX 1.1 format.

The successful chain is:

1	Windows PC	Runs BlueStacks and Command Prompt
2	BlueStacks 5 / Android 11	Runs CFMOTO RIDE
3	CFMOTO RIDE	User signs in and opens the desired historical track
4	ADB logcat	Records the app's Android log output
5	Ride API response	Contains the selected ride's trajectory
6	Trajectory parser	Keeps only valid longitude/latitude track records
7	GPX 1.1 file	Imports into onX, Garmin tools, GPX viewers, etc.

No proxy is required for this method. During the successful test, logcat itself exposed the CFMOTO API logging and trajectory. This makes the repeatable procedure substantially simpler.

2. Install BlueStacks 5 and Create the Correct Android Instance

Install BlueStacks 5 from the official BlueStacks website. BlueStacks officially supports creating an Android 11 instance through Multi-instance Manager.

[Official BlueStacks 5 download](#)

After installation, open **Multi-instance Manager** and choose **Instance -> Fresh instance -> Android 11**.

Recommended settings used successfully:

Android version	Android 11
CPU	4 cores
Memory	4 GB
Resolution	1600 x 900, Landscape
ABI	x86 & ARM
Performance	Balanced
DPI	240

Click **Download/Create**, then start the new Android 11 instance. Do not use a Nougat 32-bit instance for this procedure; Android 11 was the configuration that worked cleanly in the successful test.

[Official BlueStacks Android 11 instructions](#)

3. Install CFMOTO RIDE and Enable ADB

Inside the Android 11 BlueStacks instance, open Google Play, sign in, install **CFMOTO RIDE**, launch it, and sign in to the CFMOTO account that owns the historical ride you want to export.

[CFMOTO RIDE - Google Play](#)

Verify that you can open the app's historical ride/track list and display the desired ride on the map before continuing.

Enable Android Debug Bridge (ADB)

In BlueStacks Settings, enable **Android Debug Bridge (ADB)**. Depending on the BlueStacks version, the switch may be under Advanced settings. Restart the Android instance if BlueStacks requests it.

Find the ADB port

BlueStacks assigns an ADB localhost port to each running instance. The tested Android 11 instance used **127.0.0.1:5556**, but another computer may use a different port. BlueStacks Multi-instance Manager/Settings can show the instance information, and the command below will confirm connectivity once the correct port is used.

BlueStacks normally installs its bundled ADB executable here:

```
C:\Program Files\BlueStacks_nxt\HD-Adb.exe
```

Open **Command Prompt** (cmd.exe). For the examples below, replace **PORT** with the actual ADB port for the Android 11 instance.

```
"C:\Program Files\BlueStacks_nxt\HD-Adb.exe" connect 127.0.0.1:PORT
```

```
"C:\Program Files\BlueStacks_nxt\HD-Adb.exe" devices
```

A successful result should list something similar to:

```
127.0.0.1:PORT device
```

If it says **offline**, make sure the Android 11 instance is running, ADB is enabled, and you are using that instance's port.

4. Capture One Historical Ride with logcat

Create a simple working folder on the Windows C: drive. This avoids OneDrive/Desktop path problems and spaces in redirected folders.

```
mkdir C:\CFMOTO_GPX
```

Step A - Prepare the app

In BlueStacks, close CFMOTO RIDE or return to a screen before the desired historical ride detail. Do not open the target ride yet.

Step B - Clear old Android logs

```
"C:\Program Files\BlueStacks_nxt\HD-Adb.exe" -s 127.0.0.1:PORT logcat -c
```

Step C - Start recording logcat

```
"C:\Program Files\BlueStacks_nxt\HD-Adb.exe" -s 127.0.0.1:PORT logcat > "C:\CFMOTO_GPX\ride_capture.txt"
```

The Command Prompt will appear to sit there without returning to a prompt. **That is correct.** It is continuously recording Android's log.

Step D - Load the exact ride

While logcat is recording, go back to BlueStacks. Open CFMOTO RIDE, go to the historical Tracks/Rides area, select the exact ride you want, and wait until the complete route line is visible on the map. Give it several additional seconds so the API response is fully logged.

Step E - Stop recording

Return to Command Prompt and press **Ctrl+C** once. The capture file is now:

```
C:\CFMOTO_GPX\ride_capture.txt
```

This text file is the key source file. Keep it private because it can contain authentication tokens, vehicle IDs, GPS history, and other account information.

5. Verify That the Ride Data Was Captured

The tested CFMOTO RIDE build logged network activity with tags such as **RequestSignInterceptor**, **TokenInterceptor**, and **httplog**. The successful ride-detail response used a CFMOTO API host and a ride-history request, followed by JSON containing a **trajectory** value.

Use these Command Prompt searches as a quick check:

```
findstr /i "ridehistory trajectory httplog RequestSignInterceptor" "C:\CFMOTO_GPX\ride_capture.txt"
```

A useful capture should contain evidence of a ride-history request and/or the word **trajectory**. In the successful test, the selected ride request was a GET to a CFMOTO ride-history endpoint and the response contained the complete coordinate trajectory.

Typical trajectory record structure

```
longitude, latitude, speed_kmh, cumulative_distance_km, unix_timestamp
```

Records are concatenated into the trajectory string. Android logcat may wrap a record across display/log lines, so a parser must tolerate line wrapping. The GPX conversion should use only the trajectory records for the selected ride - not every number that looks like a coordinate elsewhere in the response.

Why validation matters

A first-pass parser that scans the entire API response for coordinate-looking numbers can accidentally include unrelated numeric data. That can produce a track that shoots across the ocean. The corrected method isolates the selected ride's trajectory and validates its geography, timestamps, cumulative distance, and point-to-point jumps before creating GPX.

6. Easiest Conversion Method - Give the Capture to ChatGPT or Another Capable Assistant

Upload `ride_capture.txt` to ChatGPT (or another assistant that can read the full file and create downloadable files). Then paste the following request. This wording tells the assistant exactly what the project is and, critically, how to avoid the bad-coordinate failure mode.

Copy/paste prompt:

```
This file is an Android ADB logcat capture made while I opened ONE historical ride in the CFMOTO RIDE app running in BlueStacks Android 11. Find the CFMOTO ride-history detail API response for the ride I opened. Locate the JSON trajectory for that selected ride. Extract ONLY the trajectory records belonging to that ride. The trajectory records are expected to contain longitude, latitude, speed, cumulative distance, and Unix timestamp values. Do NOT scan the entire response for arbitrary coordinate-looking numbers. Create a standard GPX 1.1 TRACK (.gpx) using the trajectory longitude/latitude points in chronological order. Deduplicate duplicate log copies/points where appropriate. Before creating the GPX, validate: 1. all coordinates form one geographically continuous ride; 2. there are no impossible outlier coordinates; 3. there are no giant point-to-point jumps; 4. timestamps are in chronological order; 5. the calculated polyline distance is reasonably close to the CFMOTO recorded ride distance. Report the number of track points, start coordinate, end coordinate, coordinate bounds, CFMOTO recorded distance if available, calculated GPX/polyline distance, and largest point-to-point jump. If validation passes, create and give me a downloadable .gpx file. If validation fails, do not create a misleading GPX; explain what failed.
```

Important for AI assistants: The goal is not to reconstruct the route from a screenshot. The authoritative source is the CFMOTO ride-history API response present in the logcat capture. Isolate the selected ride's **trajectory**, parse that trajectory only, validate it, and then serialize it as GPX 1.1.

7. Technical Conversion Rules for a Script or Developer

If a developer or AI system is implementing the converter directly, use these rules.

A	Identify selected ride	Find the ride-history detail response generated when the target ride was opened.
B	Isolate trajectory	Parse only the selected ride's trajectory field / trajectory records.
C	Parse tuples	Interpret records as longitude, latitude, speed, cumulative distance, Unix time where present.
D	Repair log wrapping	Join/recover records split by Android log line wrapping when the syntax clearly identifies one trajectory tuple.
E	Deduplicate	The same response may be printed by more than one logging tag. Deduplicate repeated records, preferably using timestamps.
F	Sort	Order track points by the ride timestamp.
G	Validate bounds	All points should remain in the geographic area of the ride. Reject obvious continent/ocean/outlier points.
H	Validate continuity	Calculate haversine distance between consecutive points and reject implausible jumps.
I	Validate distance	Compare polyline length with the CFMOTO cumulative/recorded ride distance. They should be reasonably close.
J	Write GPX	Use <code><trk><trkseg><trkpt lat="..." lon="..."><time>...</time></trkpt> ...</trkseg></trk></code> .

A clean GPX can contain only the track. Start/end waypoints and proprietary extensions are optional. For maximum compatibility, a simple GPX 1.1 track with timestamps is preferable.

Minimum GPX structure:

```
<?xml version="1.0" encoding="UTF-8"?> <gpx version="1.1" creator="CFMOTO RIDE Track Export"
xmlns="http://www.topografix.com/GPX/1/1"> <trk> <name>CFMOTO Ride</name> <trkseg> <trkpt lat="LATITUDE"
lon="LONGITUDE"> <time>UTC_ISO_8601_TIME</time> </trkpt> </trkseg> </trk> </gpx>
```

8. Final Validation Checklist

Do not distribute or import the GPX until these checks pass:

☐	Start coordinate is where the ride actually began.
☐	End coordinate is where the ride actually ended.
☐	All coordinates remain in the expected geographic region.
☐	No track segment crosses an ocean, another state/country unexpectedly, or jumps hundreds of miles.
☐	Track-point timestamps increase in chronological order.
☐	Calculated track length is close to the distance shown by CFMOTO RIDE.
☐	Largest consecutive point jump is plausible for the logging interval and vehicle speed.
☐	The GPX opens correctly in a GPX viewer or mapping application.

9. Import the Finished GPX into onX Offroad

In onX Offroad, open **My Content**, choose **Import**, and select the finished .gpx file. A GPX track imports as a Track. On the web map, use My Content -> Import -> Select from computer.

[Official onX Offroad import/export instructions](#)

After import, click **Show on Map** and visually confirm the track follows the expected roads/trails from beginning to end.

10. Troubleshooting - Working-Path Problems Only

ADB says the device is offline or cannot connect

Confirm the Android 11 BlueStacks instance is running, ADB is enabled, and the localhost port belongs to that instance. Re-run the connect command, then devices.

logcat command appears frozen

That is normal while recording. Open the ride in CFMOTO RIDE, wait for the map, then press Ctrl+C in Command Prompt.

The capture has no ridehistory or trajectory

Clear logcat again, start a new recording, then explicitly open the ride detail while recording and wait until the route has fully loaded. Make sure the CFMOTO account has Internet access and the ride is visible in the app.

The generated GPX draws a line to another continent

The converter parsed unrelated coordinate-looking numbers. Rebuild it by isolating only the selected ride's trajectory. Apply geographic bounds, chronological checks, point-to-point jump checks, and distance comparison before writing the GPX.

The GPX distance is slightly different from CFMOTO's displayed distance

A small difference is expected because the GPX polyline is calculated from sampled GPS points, while the app may use its own accumulated distance calculation. A large discrepancy indicates missing or bad points and should be investigated.

Privacy reminder

The raw logcat file may contain bearer tokens, vehicle identifiers, device information, and location history. Share it only with a trusted person/service for conversion, and delete unnecessary copies afterward.

11. One-Page Quick Reference

1. Install BlueStacks 5.
2. Create a fresh **Android 11** instance.
3. Install CFMOTO RIDE and sign in.
4. Confirm the target historical ride opens.
5. Enable BlueStacks ADB.
6. Determine the Android 11 instance's ADB localhost port.
7. Connect with HD-Adb.exe.
8. Clear logcat.
9. Start logcat redirection to a text file.
10. Open the exact CFMOTO historical ride and wait for its route to load.
11. Press Ctrl+C to stop capture.
12. Verify the capture contains ridehistory/trajectory data.
13. Upload the capture to ChatGPT or process it with a script.
14. Extract only the selected ride's trajectory.
15. Validate coordinates, continuity, timestamps, and distance.
16. Create a standard GPX 1.1 track.
17. Import the GPX into onX and visually verify it.

Generic command set

```
mkdir C:\CFMOTO_GPX "C:\Program Files\BlueStacks_nxt\HD-Adb.exe" connect 127.0.0.1:PORT "C:\Program Files\BlueStacks_nxt\HD-Adb.exe" devices "C:\Program Files\BlueStacks_nxt\HD-Adb.exe" -s 127.0.0.1:PORT logcat -c "C:\Program Files\BlueStacks_nxt\HD-Adb.exe" -s 127.0.0.1:PORT logcat > "C:\CFMOTO_GPX\ride_capture.txt" REM After the ride is fully displayed in CFMOTO RIDE, press Ctrl+C. findstr /i "ridehistory trajectory httplog RequestSignInterceptor" "C:\CFMOTO_GPX\ride_capture.txt"
```

Key rule: The final GPX must be created from the selected ride's CFMOTO trajectory only, followed by geographic and distance validation.

Reference links

[BlueStacks 5](#)

[BlueStacks Android 11 setup](#)

[CFMOTO RIDE on Google Play](#)

[onX Offroad GPX import](#)